Proyecto conjunto: descripción, requisitos y herramientas

EI1039 – EI1048
Grado en Ingeniería Informática

Curso 2026-2027, versión 1.0

ESTE documento presenta el proyecto conjunto de prácticas de las asignaturas *Diseño de Software* (EI1039) y *Paradigmas de Software* (EI1048) para el curso 2026/2027. El proyecto cubre las competencias de ambas asignaturas, que se complementan para la realización de un producto de software de envergadura, que sea funcionalmente operativo, que siga buenas prácticas relativas al diseño de software y de interfaces de usuarios, y que su desarrollo se ajuste rigurosamente a la metodología ágil ATDD.

El proyecto se concibe como una actividad en equipo, afectando en consecuencia a todos los aspectos y tareas atribuibles a la realización del proyecto, desde la comunicación y toma de decisiones consensuada entre los miembros del equipo, hasta el uso de herramientas para el desarrollo y gestión del software de forma colaborativa.

1. Introducción

Este documento describe el proyecto común de las asignaturas EI1039 y EI1048. Cada equipo desarrollará un proyecto de software realista, con pruebas automatizadas que garanticen la funcionalidad esperada desde el principio, lo cual os dará la oportunidad de aprender nuevas tecnologías para el desarrollo y despliegue de software. Se espera, por lo tanto, que el resultado sea un producto de software operativo y que cumpla con los requisitos que se incluyen en este documento. El profesorado de prácticas desempeñará el rol de *cliente/usuario* del producto desarrollado, por lo que cualquier cambio propuesto por un equipo en relación al proyecto descrito en este documento deberá ser aprobado por el profesorado.

2. Objetivo

El objetivo del proyecto consiste en el desarrollo de una aplicación de movilidad que permita calcular rutas entre dos lugares de interés, teniendo en cuenta el método de movilidad seleccionado por el usuario (p. ej., vehículo, bicicleta, a pie). Sobre cada ruta interesa conocer el trayecto, duración, tipo (p.ej., más rápida, más corta, etc.) y el coste asociado (€ de combustible/electricidad para rutas realizadas mediante vehículos de combustión/eléctricos, calorías para rutas en bicicleta/a pie).

La aplicación deberá contar con una interfaz gráfica de usuario a través de la cual se podrá indicar el nombre (o coordenadas geográficas) de las ciudades (o lugares) que indican el inicio y fin de una ruta, y el método de movilidad seleccionado. Para mejorar la usabilidad de la aplicación, es *obligatorio* el uso de mapas como interfaz de usuario natural para la selección del punto inicial y final de la ruta y mostrar la trayectoria calculada entre dichos puntos.

La aplicación se conectará a servicios públicos abiertos (*free open API*) para obtener la información requerida (p. ej., servicios de cálculo de rutas, geocoding, coste combustible, etc.), y deberá contar con un sistema de almacenamiento que garantice la persistencia de la información obtenida/procesada.

La lógica de negocio de la aplicación deberá estar completamente desacoplada de la interfaz de usuario, de los protocolos de comunicación y de los formatos de intercambio de datos de cada servicio. Además, como parte del diseño, se deberán considerar aquellas abstracciones y patrones de diseño de software necesarios para facilitar la implementación eficiente de la lógica de negocio. El desarrollo se realizará siguiendo la metodología ágil ATDD, la cual consiste en iteraciones determinadas por pruebas de aceptación que promueven evoluciones pequeñas y graduales del diseño a través de toda la arquitectura (*long skinny slices*), con el fin de superar progresivamente dichas pruebas (ver Sección 2 del documento de metodología y criterios de evaluación). Como el desarrollo se realizará en equipo, se empleará una metodología de desarrollo colaborativo que garantice la correcta ejecución de ATDD y del uso de herramientas de desarrollo colaborativo (ver Sección 3 del documento de metodología y criterios de evaluación).

3. Requisitos del proyecto

A continuación se especifica un conjunto de requisitos funcionales y formativos (estos últimos relativos a los resultados de aprendizaje propios de las dos asignaturas) que definen el proyecto, los cuales se consideran obligatorios para aprobar el proyecto con una nota de hasta 8 puntos. Este rango permitirá reconocer diferentes niveles de calidad en el resultado final –que incluye tanto el software desarrollado como la memoria final y presentación del proyecto– entre aquellos proyectos que satisfagan todos los requisitos, tanto funcionales como formativos. En general, se deberán mantener altos estándares de calidad en cualquier aspecto del proceso de desarrollo y del producto final.

Notas superiores a 8 *no* requerirán innovar en el alcance funcional de la aplicación, sino más bien en aspectos metodológicos y colaborativos, como los que se proponen en la sección 3.3.

3.1. Requisitos funcionales

Los requisitos funcionales que debe cumplir la aplicación y las posibles historias de usuario que pueden extraerse de los mismos son los siguientes:



Las siguientes historias de usuario extraídas de los requisitos no están escritas en piedra. Es decir, se pueden añadir historias, eliminarlas o modificar las ya existentes, siempre con el visto bueno del profesor de prácticas.

- **R1 – Gestión de usuarios:**
  - Como usuario no registrado en la aplicación quiero poder registrarme en la misma para poder utilizar sus servicios.
  - Como usuario registrado quiero iniciar sesión en la aplicación para utilizar sus servicios.
  - Como usuario quiero poder cerrar la sesión de mi cuenta para salir del sistema.
  - Como usuario quiero poder eliminar mi cuenta para que se eliminen mis datos asociados cuando ya no quiero usar el sistema.
  - ...
- **R2 – Gestión de lugares de interés:**
  - Como usuario quiero poder dar de alta un lugar de interés usando sus coordenadas para poder usarlo en una ruta.
  - Como usuario quiero poder dar de alta un lugar de interés usando su topónimo para poder usarlo en una ruta.

  - Como usuario quiero poder consultar la lista de lugares de interés dados de alta.
  - Como usuario quiero poder eliminar un lugar de interés cuando no ya no voy a utilizarlo más.
  - ...

- **R3 – Gestión de vehículos:**
  - Como usuario quiero poder dar de alta un vehículo para poder emplearlo como método de transporte en mis rutas.
  - Como usuario quiero poder consultar la lista de vehículos dados de alta.
  - Como usuario quiero poder eliminar un vehículo cuando ya no voy a utilizarlo más.
  - Como usuario quiero poder cambiar los datos de un vehículo para poder actualizar su información.
  - ...
- **R4 – Gestión de rutas:**
  - Como usuario, dados dos lugares de interés (deben estar dados de alta para poder calcular la ruta entre ellos) y un método de movilidad, quiero obtener una ruta (trayecto, distancia y duración) entre ambos lugares para visualizarla en un mapa.
  - Como usuario quiero conocer el coste asociado a la realización de una ruta en coche (€ de combustible) para saber cuánto me va a costar.



> El coste asociado a una ruta puede calcularse empleando distintas aproximaciones, algunas más simples y otras más complejas. Una forma simple sería:
> $Coste(€) = \frac{distancia(km)}{100} * consumo_medio(l/100) * precio_gasolina(€/l)$
> $Coste(€) = \frac{distancia(km)}{100} * consumo_medio(kWh/100) * \frac{precio_elec(€/MWh)}{1000}$

  - Como usuario quiero conocer el coste asociado a la realización de una ruta a pie o en bicicleta (calorías) para saber cuánto esfuerzo me va a costar.
  - Como usuario quiero conocer la ruta más rápida/corta/económica entre dos puntos.
  - Como usuario quiero poder guardar una ruta para visualizarla más adelante.
  - Como usuario quiero poder consultar el listado de rutas guardadas.
  - Como usuario quiero poder eliminar una ruta guardada cuando ya no me resulta de utilidad.
  - ...
- **R5 – Gestión de preferencias:**
  - Como usuario quiero poder marcar como favorito cada elemento de la aplicación (lugares de interés, vehículos y rutas) para que aparezcan los primeros cuando se listan.
  - Como usuario quiero establecer un vehículo/modo de transporte por defecto para el cálculo de rutas para no tener que indicarlo a mano cada vez.
  - Como usuario quiero establecer un tipo de ruta (más rápida, más corta, más económica) por defecto en el cálculo de rutas para no tener que indicarlo a mano cada vez.
  - ...
- **R6 – Persistencia de los contenidos de la aplicación (p.ej., lugares de interés, vehículos, rutas guardadas, etc.):**
  - Como usuario quiero que cada vez que inicie la aplicación, sus contenidos y aspecto sean idénticos a los que había la última vez que se cerró, con el fin de evitar reconfigurarla en cada uso.

- Como usuario quiero que la historia anterior se cumpla aunque el cierre de la aplicación haya sido involuntario (p. ej. un corte de luz).
- ...

3.2. Requisitos formativos

Al margen de los requisitos funcionales, el desarrollo del proyecto deberá ajustarse a los siguientes requisitos formativos, los cuales comprenden aspectos relacionados con los resultados de aprendizaje de las asignaturas tanto en el plano tecnológico como en el conceptual:

- El proyecto se desarrollará siguiendo la metodología ATDD (ver Sección 2 del documento de metodologías y criterios de evaluación).
- En conjunción con el punto anterior, cada equipo deberá seguir escrupulosamente las prácticas metodológicas de desarrollo colaborativo que se indican en la Sección 3 del documento de metodologías y criterios de evaluación.
- Se definirán interfaces (abstracciones) para modelar las interacciones entre la aplicación cliente y cada servicio público de información. Esto permitirá probar funcionalmente la aplicación con independencia del estado de los servicios (prueba de integración).
- Se definirá una interfaz para garantizar la independencia de la aplicación cliente con respecto a cualquier solución de almacenamiento adoptada para preservar el estado de una sesión.
- Se definirán interfaces para garantizar la independencia de la aplicación cliente con respecto a los formatos de intercambio de datos y protocolos de comunicación usados por las API.
- Se deberá diseñar e implementar una arquitectura desacoplada que garantice la independencia entre la interfaz de usuario, la lógica de negocio, la base de datos y las dependencias externas (API).
- Se deberán implementar patrones de diseño de software, justificando adecuadamente su uso, para la mejora de la eficiencia y mantenibilidad del código del proyecto.
- Se deberán implementar, justificando adecuadamente su uso, los patrones de diseño para las interfaces de usuario (MVC o similar).
- Se deberá justificar la usabilidad de la interfaz de usuario del proyecto.

3.3. Aspectos no funcionales adicionales

Como se ha comentado antes, para optar a notas superiores a 8 puntos debéis innovar en determinados ámbitos del proyecto, relacionados principalmente con buenas prácticas de desarrollo de software colaborativo. Recordad no tomar ninguna decisión de calado sin el visto bueno del profesorado de prácticas.

A continuación se proponen, a modo de ejemplo, algunas ideas que podéis explorar para extender el proyecto. Es decir, *no* es obligatorio usarlas, cada equipo puede proponer otras, aunque se recomienda enormemente explorar las ideas que se proponen a continuación.

- Buenas prácticas en repositorios como añadir un README[1] a modo de documentación minimalista del proyecto, incluyendo una breve descripción del objetivo del proyecto, un diagrama de la arquitectura, cómo instalarlo localmente y desplegarlo, LICENCIA, etc.
- Uso de CI (*Continuos Integration*) en el repositorio de Github de vuestro proyecto. Cada vez que se crea una PR (*pull request*) con los cambios de una historia, se ejecutan automáticamente las pruebas. Hay

[1] https://docs.github.com/es/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes

varias opciones, como Azure Pipelines[2] o Github Actions[3]. Requiere de leer la documentación y un poco de investigación y pruebas.

- Uso herramientas de gestión, planificación y seguimiento del trabajo. Una opción son los tableros kanban[4]. También se pueden usar Github Projects[5] donde se den de alta las historias de usuario como "issues", se asignen a los miembros (quién desarrolla, quién revisa, etc.), su estado ("todo", "in progress", etc...). Resulta muy útil enlazar en los PR a las historias de usuario ("issue") que se implementan para conseguir un seguimiento pormenorizado del progreso del proyecto.
- Hospedar el servidor, base de datos en la nube e incluso desplegar la aplicación final. Una opción ampliamente utilizada es Firebase[6] (p. ej., Firebase Cloud Firestore, Firebase Hosting, etc.), ya que cuenta con planes gratuitos para la gran mayoria de sus servicios.
- Diseñar una imagen de marca o identidad visual propia del proyecto, incluyendo por ejemplo logotipo, paleta de colores para los elementos gráficos, elementos tipográficos, etc.

4. Recursos y herramientas

4.1. Ejemplos de API

A continuación se incluye una lista de API públicas que podrás usar directamente en el proyecto:

- Cálculo de rutas [Openrouteservice[7]]:
  - Permite calcular las rutas entre dos puntos dadas sus coordenadas en función del método de movilidad empleado (p. ej., a pie, en coche, etc.). Permite obtener la ruta más rápida/recomendada/corta entre dos puntos.
  - El mismo servicio tiene una API de *geocoding* (obtención de coordenadas de una ubicación).
  - Requiere API key, registro gratuito.
  - Permite 2000 cálculos de ruta y 1000 peticiones de *geocoding* al día.
- Precio de carburantes [datos.gob.es[8]]:
  - Servicio del gobierno con los precios actualizados de los carburantes en España.
  - Proporciona un servicio REST[9] que permite obtener el precio del combustible de las gasolineras españolas, permitiendo filtrar por municipio, provincia, etc.
  - Totalmente abierto, no requiere de API key.
  - Frecuencia de actualización: 30 minutos (a XX:00 y XX:30).
- Precio de la electricidad [Red Eléctrica de España[10]]:
  - Esta API es muy completa pero un poco compleja a la hora de obetner información.
  - Totalmente abierto, no requiere de API key.

[2] https://azure.microsoft.com/es-es/services/devops/pipelines/
[3] https://docs.github.com/es/actions
[4] https://kanbantool.com/es/tablero-kanban
[5] https://docs.github.com/es/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects
[6] https://firebase.google.com/docs
[7] https://openrouteservice.org
[8] https://datos.gob.es/es/catalogo/e05068001-precio-de-carburantes-en-las-gasolineras-espanolas
[9] https://sedeaplicaciones.minetur.gob.es/ServiciosRESTCarburantes/PreciosCarburantes/help
[10] https://www.ree.es/es/apidatos

- Un ejemplo de petición para obtener los datos del precio de la luz es el [siguiente[11]].
- La consulta anterior devuelve dos tipos de precio: "Precio de mercado peninsular en tiempo real" y "Precio mercado spot" para un periodo de tiempo determinado (1 de septiembre de 2025). Los precios del primer tipo es suficiente para el proyecto. Modificad los parámetros `start_date` y `end_date` en función del día en el que se haga la consulta.



Cuando se accede a un recurso externo que proporciona información y cuya frecuencia de actualización es relativamente baja (> 15 minutos), es interesante implementar algún mecanismo de caché para evitar realizar peticiones innecesarias.



Podéis encontrar ejemplos de peticiones a las APIs y las respuestas correspondientes en el Aula Virtual o los siguientes enlaces:

- Rutas:
  - Ruta en bici entre dos puntos.
  - Ruta en coche más corta entre dos puntos.
- Geocoding.
- Precio de carburantes (respuesta limitada a primeras entradas).
- Precio de la luz.

4.2. Herramientas

A continuación aparece una lista de herramientas recomendadas en función del tipo de aplicación que queráis desarrollar y otros aspectos generales:

- **Aplicación web (frontend):** existen infinidad de *frameworks* de desarrollo para aplicaciones web, aunque quizás los más conocidos son Angular[12], Vue.js[13] o React[14]. Cualquier opción es válida, aunque en cuanto a lenguaje os recomendamos usar Typescript[15]. Como *framework* de pruebas podéis emplear Jasmine[16] o Jest[17]. Como IDE os recomendaríamos usar WebStorm[18] (con licencia educativa[19]) o Visual Studio Code[20] (gratuito) con los *plugins* necesarios para trabajar con el *framework* que seleccionéis.
- **Aplicación móvil (frontend):** *grosso modo* podemos decir que existen dos formas de desarrollar aplicaciones móviles:
  - **Lenguajes nativos:** la aplicación se desarrolla con las herramientas que proporciona la propia plataforma. Es decir, Java o Kotlin en Android, y Objective-C o Swift en iOS. Para una aplicación Android

[11] https://apidatos.ree.es/es/datos/mercados/precios-mercados-tiempo-real?start_date=2025-09-01T00:00&end_date=2025-09-02T00:00&time_trunc=hour
[12] https://angular.io/docs
[13] https://vuejs.org/
[14] https://es.reactjs.org/docs/getting-started.html
[15] https://www.typescriptlang.org/
[16] https://angular.io/guide/testing
[17] https://jestjs.io/
[18] https://www.jetbrains.com/es-es/webstorm/
[19] https://www.jetbrains.com/es-es/community/education/
[20] https://code.visualstudio.com/

lo más recomendable sería usar Kotlin junto con Jetpack Compose[21] y emplear Android Studio[22] como IDE. Para iOS, Swift[23] y Xcode[24].



No se recomienda el desarrollo de una aplicación iOS debido a que se requiere un ordenador MacOS, lo cual no es accesible para todos los bolsillos.

- **Framework multiplataforma:** herramientas que permiten desarrollar con un único código base, aplicaciones para distintos dispositivos (p. ej., Android y iOS) e incluso distintos tipos de aplicaciones (p. ej., móvil, escritorio y web). Hay *frameworks* que, por ejemplo, permiten desarrollar aplicaciones para Android y iOS usando tecnologías web como Angular. Algunos ejemplos serían Flutter[25], Ionic[26] o NativeScript[27].



La idea de crear varios tipos de aplicaciones es atractiva, pero puede conllevar muchos problemas. Para más información consultar con el profesor de prácticas.

- **Servidor (backend):** para desarrollar el servidor podéis emplear cualquier lenguaje que deseéis: Java, C#, Go, Typescript, etc., siempre y cuando dicho lenguaje cuente con *frameworks* de pruebas adecuados (p. ej., creación de *mocks*). También podéis optar por una aproximación *serverless* utilizando Firebase, por ejemplo. Si os decantáis por desarrollar una aplicación móvil, la opción de usar Firebase como backend *serverless* es muy recomendable.
- **Trabajo colaborativo:** se recomienda el uso de Git[28] como herramienta de control de versiones junto con GitHub[29] para la gestión remota del repositorio. También podéis explorar las funcionalidades que ofrece GitHub como los GitHub Issues[30] o GitHub Projects[31].
- **Librería de mapas:** un componente muy importante de la aplicación desarrollada serán los mapas. Si desarrolláis una aplicación web os recomendamos enormemente el uso de Leaflet[32]. En caso de desarrollar una aplicación móvil, Mapbox[33] es una gran opción.



Vale, pero trabajar con mapas parece ser difícil, ¿no? Aunque parezca complejo, usar mapas es tan simple como importar una librería y con un par de líneas de código ya podéis mostrar un mapa. Funcionalidades como mover el mapa, el *zoom*, etc. vienen ya implementados.
¿Obtener las coordenadas de un punto al hacer *click*? Se consigue añadiendo un simple *listener* al mapa.
¿Mostrar una ruta? La librería proporcionará un método para dibujarlas.

- **GeoJSON**[34]: no es una herramienta como tal, sino un formato de codificación para estructuras de datos geográficas. Del mismo modo que JSON se emplea para codificar y enviar información estructurada, GeoJ-

[21] https://developer.android.com/jetpack/compose
[22] https://developer.android.com/studio
[23] https://www.apple.com/es/swift/
[24] https://developer.apple.com/xcode/
[25] https://flutter.dev
[26] https://ionicframework.com
[27] https://nativescript.org
[28] https://git-scm.com
[29] https://github.com/
[30] https://github.com/features/issues
[31] https://docs.github.com/en/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects
[32] https://leafletjs.com
[33] https://www.mapbox.com/mobile-maps-sdk
[34] https://geojson.org

SON se emplea para codificar y enviar información estructurada **geográfica** como ubicaciones (*Point*) o rutas (*LineString*). Al ser un estándar, tanto Openrouteservice como las librerías de mapas trabajan con GeoJSON, por lo que su integración es sencilla.



Las herramientas y tecnologías presentadas son orientativas. Podéis emplear cualquier otra herramienta, aunque deberéis informar al profesor de prácticas para comprobar que la viabilidad del proyecto no se ve comprometida.